

O'REILLY®

Building Agent Skills for Claude Code





Elevator Pitch

5 min

Why am I teaching this?

I have been designing products for 18 years, now building AI systems for healthcare. Tools can disappear at any moment!

What does not disappear are the patterns. I noticed something: the people getting extraordinary results were not smarter or more technical. They would figure out how to teach the AI their context (their rules, their style, their domain expertise) in a way that compounds. They built skills. And skills are just files in a folder.

But the thinking is a transferable craft.

If Claude Code disappeared tomorrow, you would take these patterns to Gemini CLI or to Codex etc...

That is why I am teaching this.



Meet Your Instructor

2 min

Bence Csernak

- Agentic Engineer and Co-founder at brandguideAI and MedHubAI
- Member of the Board at Agentics Foundation and London Chapter Co-Founder
- Specializes in multi-agent orchestration
- 18+ years product design (EY, Nortal, HSBC, Publicis, GE Money Bank, Liberty Global, Bank of New York Mellon)



[linkedin.com/in/bencecsernak](https://www.linkedin.com/in/bencecsernak)



Today's Journey

Course Schedule

- **Section 1:** Why Skills Matter (25 min)
- **Section 2:** Anatomy of a Skill (35 min)
- BREAK (10 min)
- **Section 3:** Design Before You Build (40 min)
- **Section 4:** Build and Break (45 min)
- BREAK (10 min)
- **Section 5:** Beyond the Basics (25 min)
- **Section 6:** Adapting When Things Change (20 min)



Learning Outcomes

By the end of this session, you will be able to:

- Design Skill architectures that handle ambiguous inputs and edge cases gracefully.
- Build working Skills using SKILL.md conventions, file templates, and tool orchestration.
- Debug and iterate on Skills using systematic testing approaches.
- Evaluate when a custom Skill is the right solution.



SAME PROMPT, DIFFERENT RESULT

Technical — commit message

Without a skill: `fixed stuff`

With a commit-message skill: `fix(auth): handle expired refresh tokens on silent re-login` — explains why, in your team's format.

Non-technical — support reply

Without a skill: "Sorry for the inconvenience, we will look into it."

With a support-voice skill: "Sorry this happened — I can see your order shipped to the wrong address. I have started a reshipment, it will arrive by Friday, and I have refunded the difference."

Non-technical — landing page headline

Without a skill: "Welcome to Our Platform — The Best Solution for Your Business Needs"

With a brand-voice skill: "Stop losing customers to slow checkout. Start today."

What You Will Need

Prerequisites

- Claude Code CLI installed (or Claude Desktop app or `Claude.ai/code` in the browser)
- Not set up? Follow along in Codex CLI, Antigravity CLI, or OpenCode — same SKILL.md.
- **OpenCode = no lock-in:** open-source, any model provider, identical skill files.





Why Skills Matter

SECTION 1

Skills are conceptually really simple. It is just a file in a folder.

Extend what Claude knows about YOUR work, your style, your context, your subject matter expertise, the way you like it.

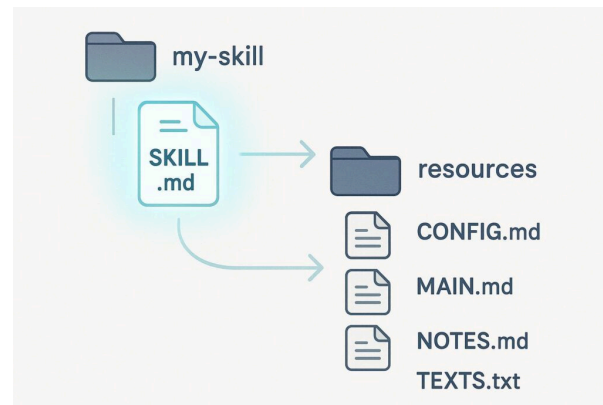
If you do something more than once a day, turn it into a skill.

What Is a Skill?

A skill is a folder containing:

- Instructions (what to do, how to behave)
- Documents (context, references, examples)
- Optional output format, for example code (for deterministic operations)

Claude loads it automatically when relevant.



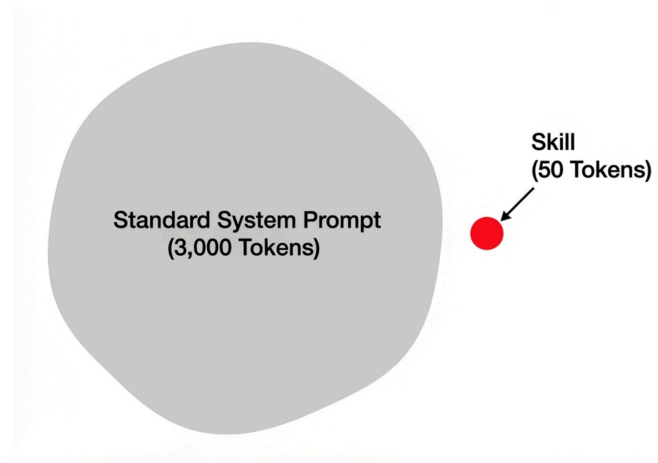


The Context Engineering Problem

The problem skills solve

- You typically load ALL tool schemas into the system prompt.
- But at any given moment, you might use only 10% of those tokens.

MCP with 20 tools = ~3,000 tokens loaded
Skill with same functionality = ~50 tokens initially





Why Skills Are Powerful

Key Benefits

- Packaging expertise into reusable modules
- Build once, use across ALL Claude products
- Progressive disclosure (loads only what is needed)
- Composable (multiple skills stack automatically)
- Portable — an open standard since Dec 2025: the same SKILL.md runs across Codex, Gemini, Copilot, Cursor and OpenCode, not just Claude

The Skill Rule

**"If you do something more than once a day,
turn it into a skill."**

Boris, the developer of Claude Code

- Coding conventions? Skill.
- Compliance checks? Skill.
- Deployment patterns? Skill.
- Your opinionated stack? Skill.





Real Use Cases

Examples in the wild

- **PDF Editor** — Rotating, editing with scripts
- **Image Editor** — Photo editing, enhancements
- **Frontend Builder**
- **Overengineering Therapist**
- **Brutally Honest Review**
- Industry providers (Supabase, Convex, Vercel, Cloudflare)



POLL: What repetitive task do you wish Claude Code just knew?

Choose the task you would most want to automate:

- Code style / conventions
- Testing patterns
- Deployment workflows
- Documentation standards
- Other (tell us in chat!)

Q&A

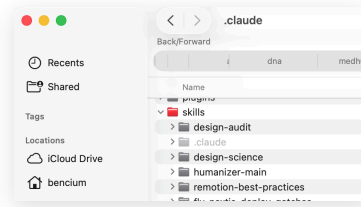
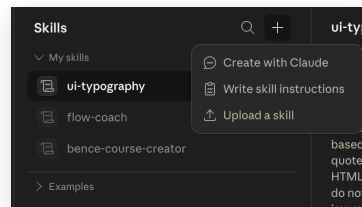
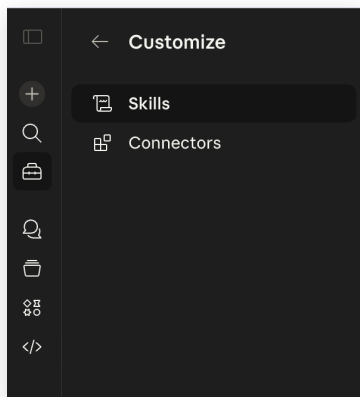
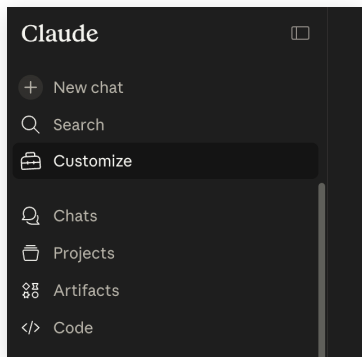




Anatomy of a Skill

SECTION 2

"The syntax may change. These patterns will not."





Skills vs Slash Commands vs MCPs

Comparison

Feature	Skill	Slash Command	MCP
Who triggers	Agent decides	You trigger	Always loaded
Context load	Name + desc only	Full prompt	Full tool schemas
Complexity	Just markdown	Predefined prompt	Server infra
Autonomy	High	Low	Medium

"By leveraging skills over slash commands, you offload more from yourself to the agent and give it more autonomy."

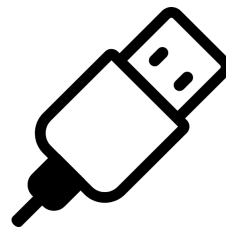
Claude's Vocabulary



SKILL = HOW something gets done — a written playbook Claude follows



SUBAGENT = WHO does it — a specialist with its own fresh context (reviewer, tester, researcher)



MCP = WHAT it connects to — a live tool or data source the agent can call



Skills vs Subagents

Key Differences

Aspect	Skill	Subagent
Execution	Inline, blocks convo	Background, async
Context	Full conversation	Fresh context only
Control	Runs to completion	Can be killed
Output	Streams to conversation	Retrieved later

When to Use What

USE A SKILL:

- Consistent behavior across tasks
- Your conventions applied automatically
- Opinionated guidance

USE A SUBAGENT:

- Parallel work (multiple reviewers)
- Long-running background tasks
- Specialized personas

USE CASES

- Enforcing your team's commit-message format automatically
- Running a full accessibility pass as a background job while you keep coding
- Reviewing a PR in your voice, without you writing the review
- Onboarding a new teammate's Claude Code to your conventions in one file

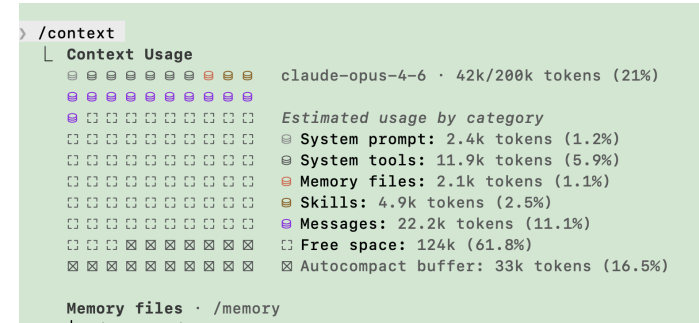


Progressive Disclosure Explained

When you give a skill to an agent:

- **STEP 1:** Only NAME + DESCRIPTION loaded into system prompt
- **STEP 2:** Agent decides if skill is relevant
- **STEP 3:** If yes, agent reads the full skill body
- **STEP 4:** Additional files loaded only as needed

Result: Massive token savings



Three Pillars of Every Skill

Every effective skill has three pillars:

1. **CONTEXT INJECTION** — What does the agent know?
2. **INSTRUCTION CLARITY** — How should it behave?
3. **OUTPUT CONVENTIONS** — What does it produce?

Master these. The rest is syntax.

Pillar 1: Context Injection

What information does the agent need?

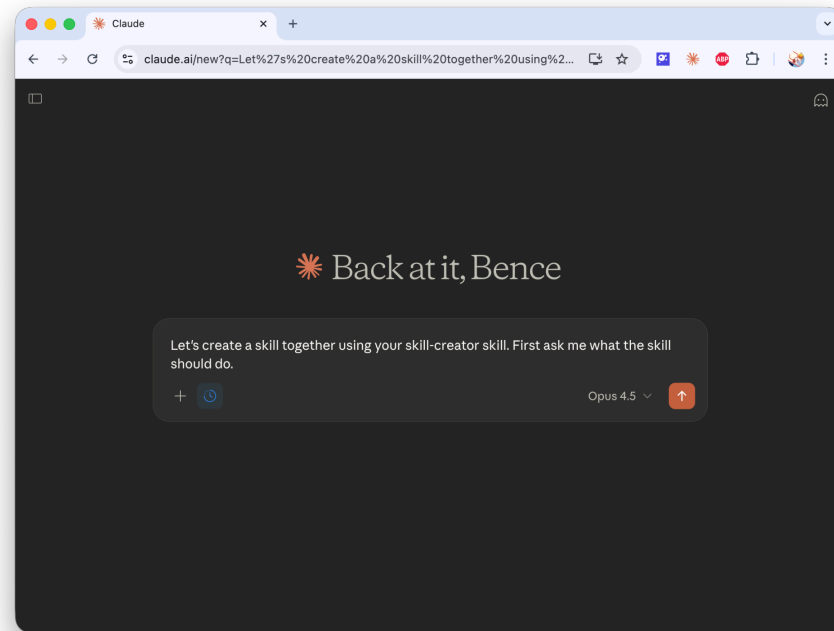
- Project conventions (your .eslintrc, style guide)
- Domain knowledge (your business rules)
- Reference files (templates, examples)
- External context (docs, APIs)





Chat Prompt

Type one thing your Claude Code does not know about your project.



Pillar 2: Instruction Clarity

How should it behave?

- Explicit boundaries ("do X, never do Y")
- Failure modes ("if unclear, ask")
- Decision rules ("prefer X over Y when...")
- Tone and style

Pillar 3: Output Conventions

What should it produce?

- Format (markdown, code, structured data)
- Location (which file, which directory)
- Verification (how to check success)
- Edge cases (what if input is weird?)



LIVE DEMO: Dissecting a Real Skill

Let us identify all three pillars in action.

- Dissecting: commit-message-skill
- Watch for: Context, Instructions, Output conventions

The screenshot shows a GitHub repository page for 'commit-message-storyteller' by 'awesome-copilot'. The page is titled 'SKILL.md' and shows a preview of the file. The preview content includes a table with the skill's name and description, followed by sections for 'Commit Message Storyteller', 'When to Use This Skill', 'Prerequisites', and 'How It Works'.

name	commit-message-storyteller
description	Analyzes git diffs or staged changes and generates narrative commit messages that explain WHY a change was made, not just what changed — following Conventional Commits format. Use when asked to "write a commit message", "generate a commit", "describe my changes", "what should I commit this as", "commit this", "summarize my diff", or "help me commit". Works with git diff output, staged files, or plain descriptions of changes.

Commit Message Storyteller

Transforms raw git diffs and change descriptions into clear, story-driven commit messages that follow the [Conventional Commits](#) specification. Instead of "update file.js", you get messages that communicate intent, context, and impact.

When to Use This Skill

- User says "write a commit message", "help me commit", or "generate a commit"
- User pastes a git diff or describes code changes
- User says "what should I commit this as?" or "summarize my diff"
- User wants better commit history for their team or open-source project
- User is preparing a pull request and wants meaningful commit messages

Prerequisites

Have at least one of the following ready:

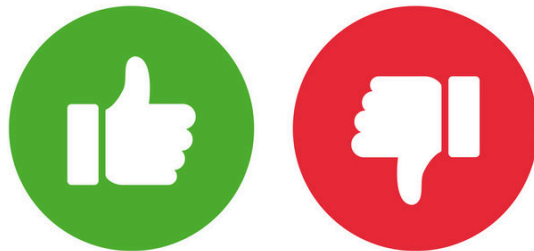
- Output from `git diff` or `git diff --staged`
- A description of what you changed and why
- A list of modified files

How It Works

Pulse Check

Are the three pillars clear?

- 👍 **Thumbs up** if yes
- 👎 **Thumbs down** if you need clarification
- If thumbs down — what is unclear? Chat it.



EXERCISE: Identify the Pillars

5 min exercise

- Open this skill: [github.com/github/awesome-copilot · commit-message-storyteller/SKILL.md](https://github.com/github/awesome-copilot/blob/main/commit-message-storyteller/SKILL.md)
- Look at the unfamiliar skill provided
- Annotate: Which parts are which pillar?





BREAK



Design Before You Build

SECTION 3

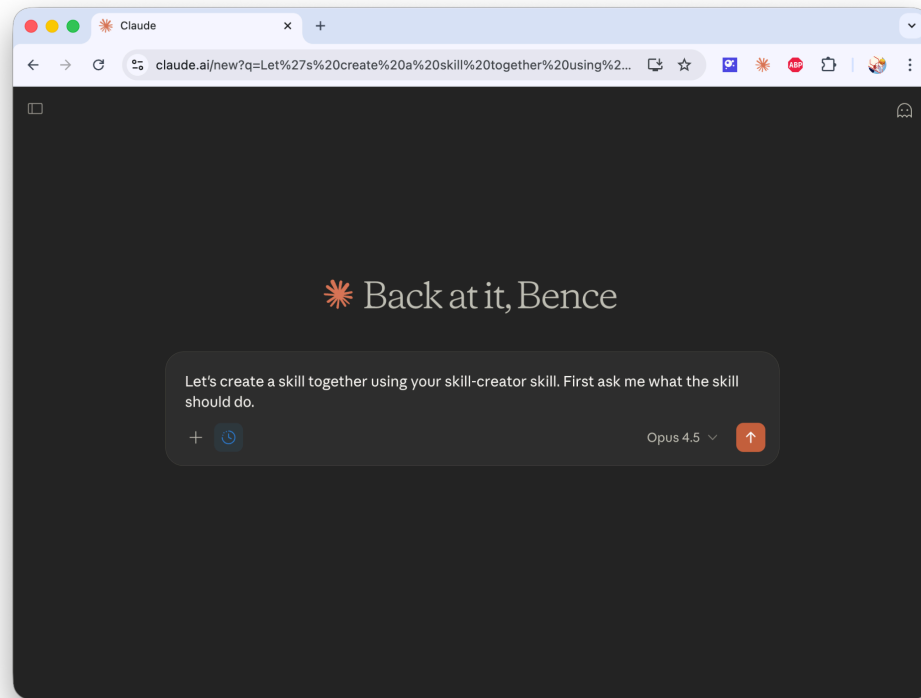
"A well-designed spec beats clever code."

The Design-First Rule

Instruct Claude.ai to write SKILL.md and interview you until all the answers are done:

1. What problem does this solve?
2. What context does it need?
3. What are the failure modes?
4. What does success look like?

Documentation-first: Write the spec before the code.



Design Pattern: Scoping

What Is In, What Is Out

This skill DOES: Generate commit messages, follow conventional commits

This skill DOES NOT:

- Actually make the commit
- Decide which files to stage

Explicit boundaries prevent scope creep.

Chat Prompt

What is ONE thing you would want your skill to explicitly NOT do?



Design Pattern: Failure Modes

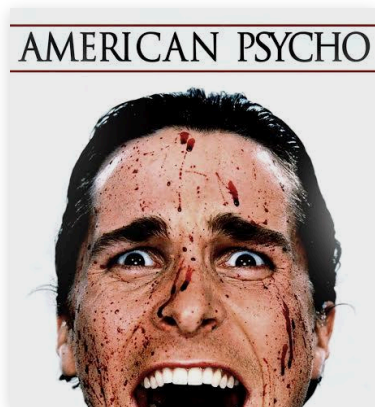
What happens when input is unclear?

- **OPTION A:** Guess confidently (risky)
- **OPTION B:** Ask the human (safe but slow)
- **OPTION C:** Graceful degradation (middle ground)

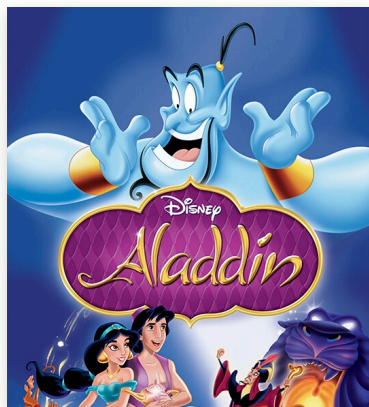
Define this BEFORE you build.

Failure Mode Analogies

LLM Failure Mode Analogies



"Patrick Bateman" =
Hallucination (confident
+ wrong)



"The Genie" = Specificity
trap (does exactly what
you asked, not what you
meant)



"Lucy Whitmore" =
memory limit (forgets
everything)

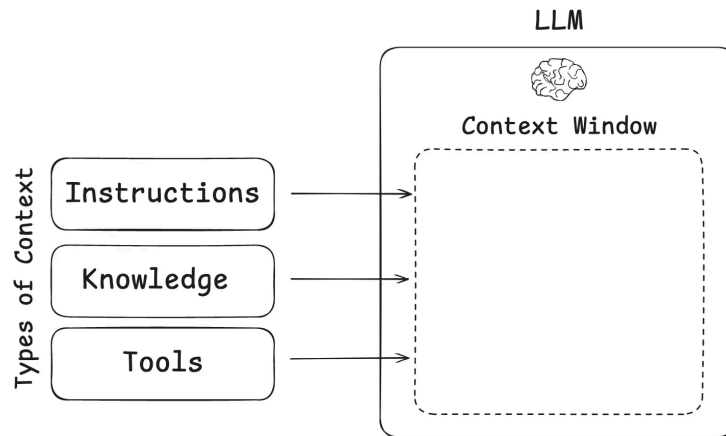
The Minimum Context Principle

Just enough information to succeed.

- **Bad:** "Here is our entire codebase history..."
- **Good:** "Here is our style guide + 2 examples."

Why?

- Token economics matter
- More context = more confusion
- Focused context = better results





EXERCISE: Design Your Skill Spec

15 min exercise

Design a skill you actually want to build. Copy this template and fill it in:

1. **Problem** — what repetitive task should the skill own?
2. **Context needed** — rules, style, domain knowledge, examples?
3. **Failure modes** — ambiguous inputs, edge cases, where it breaks?
4. **Success criteria** — how will you know it worked?

You will BUILD this in Section 4.

15



EXERCISE IN PROGRESS

10 minutes remaining

Keep working on your skill design

10



EXERCISE IN PROGRESS

5 minutes remaining

Start wrapping up your design

5



EXERCISE IN PROGRESS

Wrapping up

Finish your current thought

0



Peer Review

In chat, post ONE sentence:

"The biggest ambiguity risk in my skill architecture/design is..."

POLL: What surprised you about your design?

Select the option that best matches your experience:

- Harder to scope than expected
- More failure modes than I thought
- Needed less context than assumed
- Needed more context than assumed



Q&A





Build and Break

SECTION 4

"The only way to learn is to ship broken things."



SKILL.md File Structure

claude.ai will write for us

Standard structure:

```
---  
name: my-skill-name  
description: one-line description  
---  
  
## What This Skill Does  
[Brief explanation]  
  
## Instructions  
[The actual behavior rules]  
  
## Examples  
[Show input → output]
```



Common Pitfalls

claude.ai will write for us

Avoid these mistakes:

1. **Instructions too vague** — Bad: "Write good code" · Good: "Follow PEP-8, include type hints"
2. **Context too heavy** — Bad: "Reference the entire codebase" · Good: "Reference style-guide.md only"
3. **Missing failure handling** — Bad: "[no edge cases]" · Good: "If unclear, ask before proceeding"

TROUBLESHOOTING:

Skill not triggering?

Check name/description
keywords

Triggers too often?

Be more specific in description

Context budget exceeded?

Use progressive disclosure



LIVE DEMO: Building from Scratch

10 min demo

- The real process (including mistakes)
- How to test as you go
- What debugging looks like — test and iterate



EXERCISE: Build Your Skill

Use claude.ai or chatGPT!

20 min exercise

- Build the skill you designed in Section 3
- Start from: `demo-repo/starter-template/`
- Success: SKILL.md follows conventions, test it, verify output

20



BUILDING...

15 minutes remaining

Keep building your skill

15



BUILDING...

10 minutes remaining

Continue working on your skill

10



BUILDING...

5 minutes remaining

Start wrapping up

5



BUILDING...

Time to test!

- Test your skill with a real prompt
- Verify output matches your spec



0

When Your Skill Does Not Work

Debug workflow:

1. Tell Claude: "The skill failed because [X]"
2. Show: Expected output vs actual output
3. Ask: "Review the full skill and identify failure point"
4. Implement the fix and test again



Discussion: What broke?

Share in chat

- What failed?
- What was easier than expected?
- What would you do differently?



Q&A





BREAK

The Meta-Skill

SECTION 6

"How to adapt when the tools inevitably change"





What Is Durable

Ask the LLM when in doubt. Ask multiple LLMs.

INVEST HEAVILY IN THESE (they last):

- Context management patterns
- Instruction design principles
- Output conventions
- Mental models for agentic tools
- The three pillars framework



What Is Volatile

HOLD LOOSELY (they change):

- Specific file syntax (SKILL.md format)
- API details and endpoints
- Tool-specific features
- CLI flags and commands
- Pricing and rate limits

The Transfer Test

If Claude Code disappeared tomorrow, could you apply these patterns to:

- Codex CLI?
- OpenCode? (open-source, any model provider — no lock-in)
- The next tool that does not exist yet?

If yes, you learned the right things.

This is not hypothetical: Gemini CLI — named on these very slides — is being retired. The patterns survive; the tool did not.

Portability Is Real Now

One set of skill files, read by every major tool.

Write the skill once — every agent reads it

- `.agents/skills/` — the shared convention. **Codex CLI** and **Gemini CLI** read it natively (Gemini even prefers it over its own folder).
- **Claude Code** reads `.claude/skills/` — join the shared folder with a one-line symlink (native support is an open request).
- Per-tool paths work too: `~/.codex/skills/` · `~/.gemini/skills/` · `~/.claude/skills/` · `.opencode/skills/`

Same SKILL.md, four tools, no copies. That is the transfer test — on disk, not in theory.



Discussion

What changes and discontinued things you noticed?



POLL: What pattern will you use first?



Choose your top priority:

- The three pillars framework (Context / Instructions / Output)
- Design-before-build workflow
- The debug loop (tell Claude what failed → fix → retest)
- Progressive disclosure (name + description first, rest on demand)



Staying Current

How to stay current

1. Read changelogs pattern-first
2. Watch for capability shifts, not syntax shifts
3. Test skills on real work regularly — broken skills = learning opportunities
4. Re-run your skill's test prompts after every major model or tool update
5. Watch community skill directories (agentskills.to, smithery.ai) for patterns others are already finding

Troubleshooting

Skill not triggering

If Claude does not use your skill when expected:

- Check the description includes keywords users would naturally say
- Verify the skill appears in "What skills are available?"
- Try rephrasing your request to match the description more closely
- Invoke it directly with `/skill-name` if the skill is user-invocable

Troubleshooting

Skill triggers too often

If Claude uses your skill when you do not want it:

- Make the description more specific
- Add `disable-model-invocation: true` if you only want manual invocation

Claude Does Not See All My Skills

Skill descriptions are loaded into context so Claude knows what is available. If you have many skills, they may not all get listed.

- Run `/context` to check for a warning about excluded skills.
- Run `/doctor` to check your Claude Code installation and configuration for issues that could affect skill discovery.

Final Q&A



Key Takeaways

Remember these four things:

1. Skills = packaged expertise (HOW to do something)
2. Three pillars: Context, Instructions, Output
3. Design before you build
4. Patterns survive; syntax does not

Skills extend what Claude can do. Create a SKILL.md file with instructions, and Claude adds it to its toolkit.



Key Takeaways

- Always ask Claude **if she really sees** the skill when in doubt.
- Agent Skills is an open standard (since Dec 2025): the same **SKILL.md** runs across Codex, Gemini, Copilot, Cursor and OpenCode — see **agentskills.io**.
- **SKILL.md** file can be global in your **.claude** folder or local in your **project** folder.
- Claude Code automatically discovers skills from nested **.claude/skills/** directories.
- You are fine with one file SKILL.md

Resources

Where to continue learning:

- **Official Docs:** code.claude.com/docs/en/skills — the canonical SKILL.md spec
- **The standard:** agentskills.io · github.com/agentskills/agentskills — the open Agent Skills spec
- **Example skills:** github.com/anthropics/skills · github.com/openai/skills — official libraries
- **Community:** agentskills.to · smithery.ai/skills — skills others have published
- **Plugins:** `/plugin` bundles skills + hooks + MCP into one install
- **My marketplace:** github.com/bencium/bencium-marketplace — 18 ready-to-install skills
- Industry providers ship their own skills: Supabase, Convex, Vercel, Cloudflare

Surprising skills:

- github.com/proffesor-for-testing/agentiq — open-source QA/testing agent fleet with skills for the full SDLC
- github.com/blader/Claudeception — a skill that has Claude extract and learn new skills from its own work
- github.com/cgbarlow/skills — a small personal collection of ready-to-drop-in Claude skills
- remotion-dev/remotion/packages/skills — skills for Remotion, the "make videos with React" framework
- K-Dense-AI/claude-scientific-skills — 163 validated skills + 100+ science databases, used across biology, chemistry, medicine



Connect

Bence Csernak

<https://www.linkedin.com/in/bencecsernak>

Course feedback helps improve future sessions!

Thank you for building with me today.



The image features the O'Reilly logo in white, centered on a blue gradient background. The background transitions from a darker blue on the left to a lighter blue on the right. Large, faint, overlapping circles in various shades of blue are visible in the background, creating a sense of depth and movement. The logo itself is in a clean, sans-serif font, with a registered trademark symbol (®) at the end.

O'REILLY®